

Block, inline, and display

You have used DevTools to inspect what the browser did with your CSS. Now it is time to inspect a different kind of default: how elements naturally sit on the page before you write layout CSS.

Some elements start on a new line, some sit inside a line of text, and some can be changed when the default behavior is not what you need. That behavior is controlled by the `display` property.

Start with a mixed page

Start with an `index.html` file that mixes block and inline elements:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Display practice</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <main class="page">
      <section class="intro">
        <p class="eyebrow">CSS practice</p>
        <h1>Block, inline, and display</h1>
        <p class="summary">Some elements stack like sections. Some sit inside a sentence</p>
      </section>

      <nav class="lesson-nav" aria-label="Lesson sections">
        <a href="#card">Card</a>
        <a href="#notice">Notice</a>
        <a href="#actions">Actions</a>
      </nav>

      <article class="card" id="card">
        <span class="badge">New</span>
        <h2>Display controls how an element participates in layout</h2>
        <p class="card-text">
          A paragraph is a block element by default, but a
          <a href="/docs">link</a>
          sits inline with the sentence.
        </p>
      </article>

      <p class="notice" id="notice">
        This notice is a paragraph, so it starts on its own line and takes the available
      </p>

      <div class="actions" id="actions">
        <a class="action-link" href="/start">Start practice</a>
        <a class="action-link secondary-action" href="/notes">Read notes</a>
      </div>
    </main>
  </body>
</html>
```



Then add this stylesheet:

CSS

```
* {
  box-sizing: border-box;
}

body {
  font-family: system-ui, sans-serif;
  color: #111827;
  background-color: #f9fafb;
}

.page {
  max-width: 760px;
  margin: 40px auto;
}

.intro,
.card,
.notice {
  background-color: white;
```

```

}

.intro,
.lesson-nav,
.card,
.notice {
  margin-bottom: 24px;
}

.eyebrow {
  color: #2563eb;
  font-weight: 700;
}

.summary,
.card-text {
  color: #4b5563;
  line-height: 1.6;
}

.badge {
  background-color: #fff6ff;
  color: #1d4ed8;
  padding: 6px 10px;
  border-radius: 999px;
  font-size: 14px;
  font-weight: 700;
}

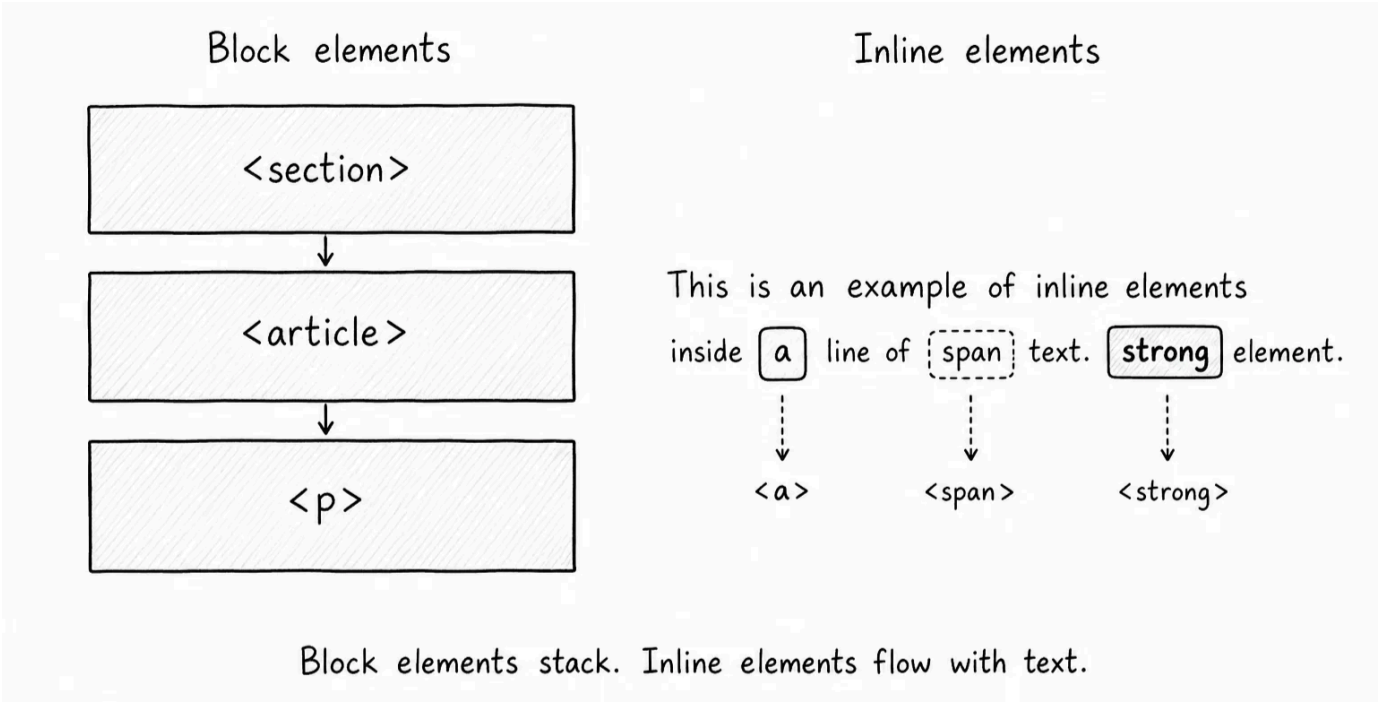
.action-link {
  width: 180px;
  padding: 12px 16px;
  color: white;
  background-color: #2563eb;
  border-radius: 6px;
  text-align: center;
  text-decoration: none;
  font-weight: 700;
}

.secondary-action {
  color: #2563eb;
  background-color: white;
  border: 1px solid #bfdbfe;
}
```

Open the page and notice a few things before changing the CSS:

- The intro, card, and notice stack vertically.
- The navigation links sit side by side because links are inline by default.
- The badge sits before the heading because `span` is inline by default.
- The action links have padding and color, but `width: 180px` does not behave the way you might expect.

Those differences are the point of this lesson.



Block elements stack

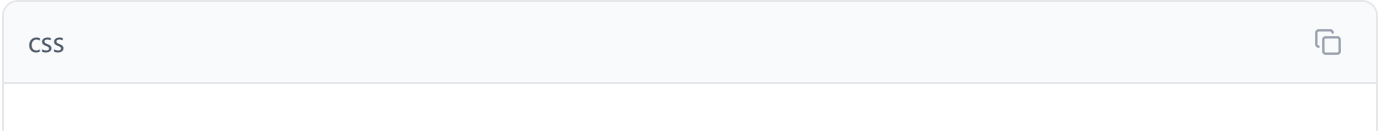
A block element starts on a new line and usually takes the full available width. Common block elements include:

- `<h1>` through `<h6>`
- `<p>`
- `<div>`
- `<section>`
- `<article>`
- `<header>`
- `<footer>`
- `<nav>`

That is why in our webpage example, the intro, card, and notice stack one after another. Their default display behavior is block-like.

Inspect the notice in DevTools. In the `Computed` panel, search for `display` ; you should see that the paragraph is `display: block` .

You can make this behavior explicit:



```
.notice {  
  display: block;  
}
```

That does not change much because the paragraph was already block by default. It just names the behavior the browser was already using.

💡 **Defaults are still CSS behavior** - Browser defaults are not magic. A paragraph stacks because the browser treats it as a block-level element by default, and DevTools can help you confirm that default instead of guessing.

Inline elements flow with text

An inline element stays inside the current line of text. Common inline elements include:

- `<a>`
- `<span>`
- `<strong>`
- `<em>`
- `<code>`

That is why the link inside the paragraph sits inside the sentence:

html

```
<p class="card-text">  
  A paragraph is a block element by default, but a  
  <a href="/docs">link</a>  
  sits inline with the sentence.  
</p>
```

The link does not start a new line. It flows with the words around it.

The badge works the same way:

html

```
<span class="badge">New</span>
```

A `span` is inline by default, so it sits near the text around it instead of becoming a full-width box.

Inline elements do not behave like full boxes

Inline elements can accept many visual styles, but they do not behave like full block boxes.

Look at the action links:

CSS

```
.action-link {  
  width: 180px;  
  padding: 12px 16px;  
}
```

The padding is visible, but `width: 180px` does not create a consistent button width because links are inline by default.

That is a common beginner surprise: `width` and `height` do not work on most normal inline elements the way they work on block elements. Some elements, like images and inputs, are exceptions. You will see those later.

Inspect an action link in DevTools. The `width` declaration may appear in the Styles panel, but the final layout still follows inline behavior. The CSS is not broken; the display type is the missing piece.

Change display when the default is wrong

The `display` property changes how an element participates in layout.

For action-style links, `inline-block` is often useful:

CSS

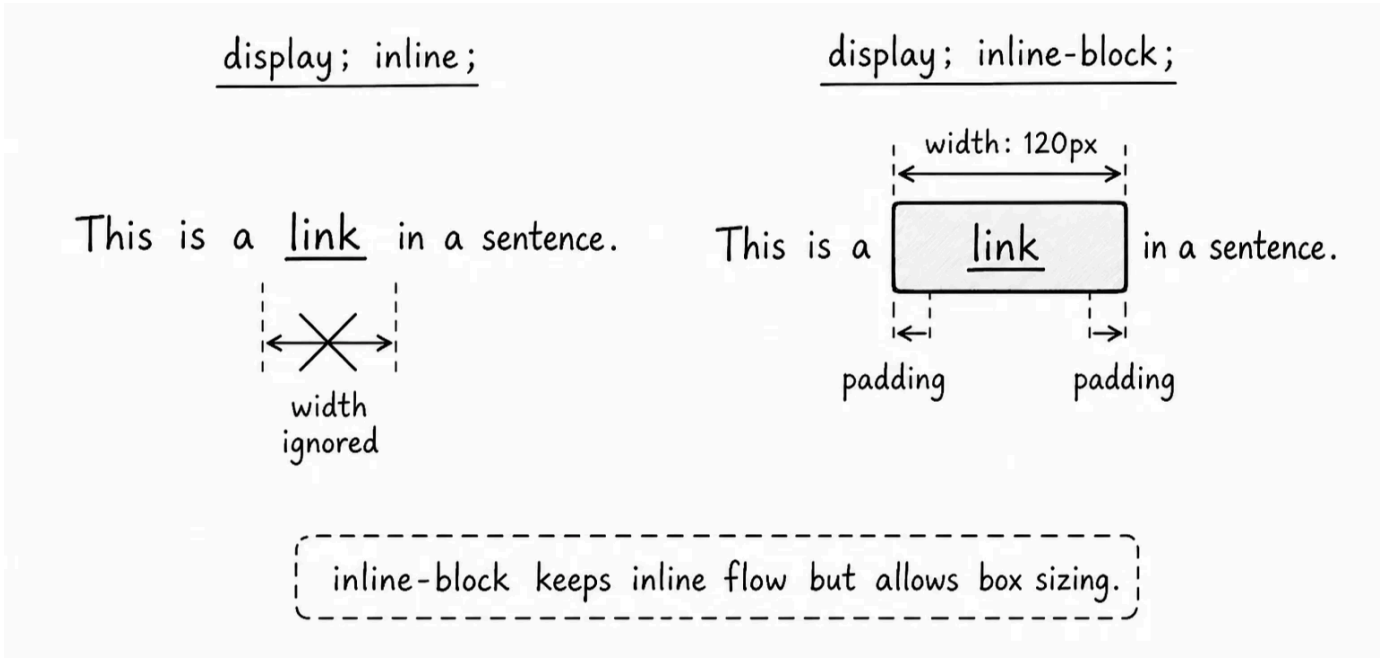
```
.action-link {  
  display: inline-block;  
  width: 180px;  
  padding: 12px 16px;  
  color: white;  
  background-color: #2563eb;  
  border-radius: 6px;  
  text-align: center;  
  text-decoration: none;  
  font-weight: 700;  
}
```

Reload the page and the two action links should now keep their button-like shape and use the width you set.

`inline-block` is a middle ground:

- It sits in a line like inline content.
- It can use `width` , `height` , padding, border, and margin more like a block box.

That makes it useful for small things like badges, labels, and simple button-style links.



Use display block for full-width actions

Sometimes you want a link to behave like a full-width block.

Try changing the action link rule:

CSS

```
.action-link {  
  display: block;  
  width: 100%;  
}
```

Now each action link starts on its own line and fills the available width.

That may be useful for a vertical menu or a stacked mobile layout, but it is not the right choice for every link. A normal text link inside a paragraph should stay inline.

For these action links, `inline-block` is usually the better fit because the links can sit side by side while still behaving like button-shaped boxes.

The useful question is: should this element behave like text, like a box, or like a full row?

Hide elements with display none

You can hide an element with `display: none`:

CSS

```
.notice {  
  display: none;  
}
```

The notice disappears, and the page closes the space where it used to be.

That is different from making the text transparent or changing the background. `display: none` removes the element from layout, so it no longer takes up space.

There is a related property worth knowing: **visibility: hidden** . It hides the element, but it leaves the space the element would have taken up. Use **display: none** when the element should not exist in the layout. Use **visibility: hidden** when it should still hold its space, but not show.

Use this carefully. If content is important for the user, hiding it can create accessibility and usability problems.

Display does not change meaning

CSS can change how an element is displayed, but it does not change what the HTML means.

For example, you can make a link look like a button:

CSS

```
.action-link {
  display: inline-block;
  background-color: #2563eb;
  color: white;
}
```

It is still a link, and it still navigates somewhere because the HTML is **<a>** .

You can also make a heading look small:

CSS

```
h1 {
  font-size: 18px;
}
```

It is still the page heading because the HTML is **<h1>** .

Screen readers, search engines, keyboard navigation, and browser features use the HTML element, not just the CSS display value. A link styled like a button is still announced as a link. A small-styled **<h1>** is still the page's top heading for navigation tools.

Choose HTML for meaning, and use CSS display for layout behavior.

 **Keep meaning and layout separate** - Do not pick **** just because you want something inline, or **<div>** just because you want something block. Pick HTML for meaning first, then use CSS when you need to change the display behavior.

Display grows into layout systems

You have already seen this rule:

CSS

```
.tag-row {  
  display: flex;  
  gap: 8px;  
}
```

You used `display: flex` in the spacing lesson to make `gap` work. That was a display value all along. It told the browser to use a flex layout mode for that parent's children.

`grid` is also a `display` value.

You do not need to learn them fully in this lesson. For now, the important idea is that `display` chooses the layout mode for an element.

- `block` stacks as a full-width box.
- `inline` flows with text.
- `inline-block` flows with text but behaves more like a box.
- `none` removes the element from layout.
- `flex` and `grid` create layout systems for an element's children.

Flexbox and grid get their own lessons later. This lesson gives you the foundation for why those layout systems start with `display`.

Try it

Now change display behavior on purpose and watch how the layout responds:

- Inspect the notice in DevTools and confirm its computed `display` value.
- Add `display: block;` to `.lesson-nav a`. Notice that each navigation link starts on its own line.
- Change `.lesson-nav a` back to inline behavior by removing `display: block;`.
- Add `display: inline-block;` to `.badge`, then add `margin-bottom: 12px;`. Notice that the badge can now create more reliable space around itself.
- Change `.action-link` from `inline-block` to `block`. Notice that each action link becomes its own row.
- Add `display: none;` to `.notice`, then inspect the page. Notice that the notice no longer takes up space.

- Replace `display: none;` with `visibility: hidden;` on `.notice`.
Notice that the notice disappears, but its space stays.

The goal is to stop treating element layout as random. If an element will not size, stack, or sit inline the way you expect, check its `display` value.

< 0 / 12 >

✔ Knew 0 Items

✦ Learnt 0 Items

⏮ Skipped 0 Items

↺ ResetProgress

Test Your Understanding

What does display control?

[Click to Reveal the Answer](#)

✔ Already Know that

✦ Didn't Know that

⏮ Skip Question

✓

LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →